Evolutionary Intelligence (2025) 18:6
https://doi.org/10.1007/s12065-024-00989-6

RESEARCH PAPER

Deep reinforcement learning-based spatio-temporal graph neural network for solving job shop scheduling problem

Goytom Gebreyesus[1] · Getu Fellek[1] · Ahmed Farid[1] · Sicheng Hou[1] · Shigeru Fujimura[1] · Osamu Yoshie[1]

Received: 1 January 2024 / Revised: 26 September 2024 / Accepted: 10 October 2024 / Published online: 16 November 2024
© The Author(s) 2024

Abstract

The job shop scheduling problem (JSSP) is a well-known NP-hard combinatorial optimization problem that focuses on assigning tasks to limited resources while adhering to certain constraints. Currently, deep reinforcement learning (DRL)-based solutions are being widely used to solve the JSSP by defining the problem structure on disjunctive graphs. Some of the proposed approaches attempt to leverage the structural information of the JSSP to capture the dynamics of the environment without considering the time dependency within the JSSP. However, learning graph representations only from the structural relationship of nodes results in a weak and incomplete representation of these graphs which does not provide an expressive representation of the dynamics in the environment. In this study, unlike existing frameworks, we defined the JSSP as a dynamic graph to explicitly consider the time-varying aspect of the JSSP environment. To this end, we propose a novel DRL framework that captures both the spatial and temporal attributes of the JSSP to construct rich and complete graph representations. Our DRL framework introduces a novel attentive graph isomorphism network (Attentive-GIN)-based spatial block to learn the structural relationship and a temporal block to capture the time dependency. Additionally, we designed a gated fusion block that selectively combines the learned representations from the two blocks. We trained the model using the proximal policy optimization algorithm of reinforcement learning. Experimental results show that our trained model exhibits significant performance enhancement compared to heuristic dispatching rules and learning-based solutions for both randomly generated datasets and public benchmarks.

Keywords Deep reinforcement learning · Spatio-temporal representation · Job shop scheduling · Graph neural network

1 Introduction

With the advent of Industry 4.0, the need for automation of manufacturing processes is progressively increasing [1]. In this regard, the application of artificial intelligence pivoted by deep learning has paramount importance in automating various conventional tasks [2]. To this end, deep learning is being widely used for solving scheduling problems in the modern manufacturing environment, demonstrating several advantages over traditional approaches [3]. The job shop scheduling problem (JSSP), a complex NP-hard combinatorial optimization problem (COP), is one of the important areas where deep learning-based approaches are proving effective. Deep learning solution approaches offer acceptable solutions for complex JSSPs within a reasonable computation time and show promise in handling high-dimensional state spaces.

In recent years, reinforcement Learning (RL) has emerged as a prominent method for solving complex COPs, including

✉ Goytom Gebreyesus
goytom.desta@fuji.waseda.jp

Getu Fellek
getutadesse@toki.waseda.jp

Ahmed Farid
ahmed.farid@asagi.waseda.jp

Sicheng Hou
sicheng_hou@fuji.waseda.jp

Shigeru Fujimura
fujimura@waseda.jp

Osamu Yoshie
yoshie@waseda.jp

[1] Graduate School of Information, Production and Systems, Waseda University, Fukuoka, Japan

the JSSP. Q-learning, the most well-known RL algorithm, is widely employed and has demonstrated successful applications in solving scheduling problems. For example, Zhang *et al*. [4] proposed Q-learning-based scheduling for dynamic parallel machine environments, minimizing mean weighted tardiness with adaptive policies. Shahrabi *et al*. [5] employed Q-learning and policy gradient methods iteratively, introducing a novel RL-based framework for parameter estimation, enhancing accuracy and computational efficiency in complex models. Kardos *et al*. [6] applied Q-learning to dynamic scheduling in a job shop production system, demonstrating the effectiveness of RL for production scheduling problems. Additionally, Zhao *et al*. [7] proposed a DQN-based scheduling algorithm for dynamic JSSP in smart manufacturing, enhancing adaptive scheduling performance. However, despite their successful applications in solving JSSPs, these methods heavily rely on handcrafted features, wherein the learned agent selects the optimal scheduling rule, which limits their scalability and adaptability [8] [9].

To address these limitations, recent advancements in the field have led to widespread adoption of sequence-to-sequence and graph-based deep reinforcement learning (DRL) frameworks. For instance, Bello *et al*. [10] and Nazari *et al*. [11] proposed recurrent neural network (RNN)-based frameworks trained using RL for solving typical COPs. Inspired by the recent evolution in deep learning, graph neural networks (GNNs) and transformer network-based frameworks are being widely used for solving COPs. These frameworks learn a graph representation to capture the structural information of the problem, which is followed by a policy network to deliver an end-to-end solution for the underlying problem.

In the context of the JSSP, problem instances can be viewed as disjunctive graphs, with nodes representing specific operations and edges indicating the order of operations on each machine. The actions taken at each decision step induce dynamics in the environment, causing changes in both the topological structure of the graph and the temporal evolution of its features. These attributes encompass spatial (structural information) and temporal (time-dependent) characteristics. Although both attributes can shape the status of the job shop, contemporary GNNs such as the graph isomorphism network (GIN) [12] and graph attention network (GAT) [13], as well as transformer-based DRL frameworks [14], primarily learn spatial features to capture structural information, without encoding the time-dependent attributes of the graph. The evolution of features as a result of actions taken is crucial for the decision-making process at the current time step [15]. Therefore, this temporal information should be integrated with the structural information of the JSSP graph to capture the full dynamics of the scheduling environment. Despite effectively capturing the structural information of the JSSP environment, we argue that including the temporal aspects enhances graph representation learning, resulting in more comprehensive graph information that accurately reflects the dynamics in the problem environment.

In this study, we presented a novel DRL framework, called Deep Reinforcement Learning-based Spatio-Temporal Graph Neural Network (DRL-STGNN). The novelty of the DRL-STGNN is twofold. First, unlike existing frameworks, the DRL-STGNN is designed to learn time-dependent aspects of the JSSP explicitly. To address the time-dependent aspect of the JSSP, we define the JSSP as a time-varying graph based on the changing features of the environment following the decisions at each time step. To capture the time-dependent attributes of the time-varying graph, we introduce a temporal block that encodes the updated node attributes at each time step. By combining both spatial and temporal information, our framework captures the interdependencies between nodes and feature evolution, enabling the extraction of expressive graph representations at each time step.

Second, the DRL-STGNN introduces an innovative GNN architecture adapted to iteratively learn rich representations by integrating both the spatial and temporal attributes of the underlying disjunctive graphs of the JSSP environment, as opposed to those in previous studies. To this effect, we first designed an innovative attentive GIN (Attentive-GIN)-based spatial block to extract the spatial attributes of the disjunctive graph. Then, we designed a separate temporal block that dynamically learns the temporal aspects of the JSSP graphs. The learned representations from the two blocks are followed by a novel gate-based fusion block to selectively integrate the learned information and construct rich and complete graph and node representations. These representations are passed to a pointer network-based decoder from which an optimal solution is constructed. We trained the DRL-STGNN using the actor-critic-based proximal policy optimization (PPO) algorithm owing to its stability and sample efficiency [16].The Results revealed that the DRL-STGNN outperforms state-of-the-art learning-based solutions and heuristic methods from generated instances to publicly available benchmarks.

The contributions of this study are summarized as follows:

1. We propose a novel DRL framework to solve the JSSP that is uniquely designed to extract and integrate both the spatial and temporal attributes of the JSSP environment for constructing rich representations. The DRL-STGNN, with its innovative GNN architecture, iteratively learns enriched representations from the underlying disjunctive graphs of the JSSP. This integration of temporal and structural attributes enables the DRL-STGNN to construct rich and complete graph representations.

2. Our study introduces a novel approach for the JSSP by encoding the time dependencies among time-varying node attributes. We model the JSSP as a time-varying graph, reflecting the evolving nature of the environment with each decision. To this end, we implement a temporal block in the DRL-STGNN, designed to encode updated node attributes at each time step.
3. As part of extracting the spatial attributes of the JSSP environment, we designed a novel Attentive-GIN that allows every node to aggregate information from its neighborhood connectivity based on the corresponding attention weights, which show the relative importance to the target node.
4. We conducted extensive comparative experiments on randomly generated test instances and two public JSSP benchmarks to evaluate the performance of the DRL-STGNN against heuristics and learning-based solutions.

The subsequent sections of this article are structured as follows: Sect. 2 reviews recent DRL-based scheduling methods. Section 3 presents a brief description of the JSSP and its disjunctive graph representation. Section 4 provides a detailed description of the proposed DRL framework. Section 5 presents experimental settings and results. Finally, Sect. 6 provides a conclusion to this article.

2 Literature review

The JSSP is a well-known example of a COP, where several jobs must be distributed across several machines while adhering to a set of constraints. The JSSP is a challenging problem due to its high computational complexity, which increases with the number of jobs and machines.

In traditional operations research, exact methods such as integer programming and the branch and bound approach were proposed to solve the JSSP [17] [18]. However, these methods face intractability with larger problems and struggle to find optimal solutions even for small instances [19]. As a more efficient alternative, metaheuristics methods have been introduced for the JSSP, which are capable of efficiently finding near-optimal solutions for large-scale problems [20]. These methods have been widely applied for solving different optimization tasks, such as path planning and energy dispatch scheduling [21]. Notable metaheuristics approaches for the JSSP include tabu search and genetic algorithms [22] [23].

The advent of RL has introduced new possibilities in solving COPs, attracting significant attention for its potential in complex decision-making environments [24]. One of the widely used RL algorithms is Q-learning, in which the agent learns the optimal policy for scheduling jobs [5] [6]. A study by Sur *et al*. [25] proposed a unified framework for optimizing trajectory planning for autonomous vehicles using constrained RL techniques. The framework integrates phase-aware architectures and trajectory optimization algorithms, demonstrating improved traffic efficiency and safety. Recent advancements have further expanded the scope of RL methodologies. Ding et al. [26] presented a novel asynchronous RL algorithm, utilizing enhanced parallel particle swarm optimization (PSO) techniques. The algorithm aims to improve the efficiency of RL processes by leveraging advancements in parallel PSO. Another study by Zhang et al. [27], introduced an offline RL method with augmented data (ORAD) framework. The framework employs Q-value regularization in offline RL settings, offering potential enhancements to the stability of Q-learning by addressing value overestimation problems.

DRL, combining deep learning with RL, has emerged as a significant area of research in artificial intelligence. This approach leverages deep neural networks to solve problems where agents learn to make decisions by interacting with a complex environment. In this context, the work by Zhao et al. [28] introduced a dynamic weight updating mechanism for asynchronous DRL systems, which adapts learning parameters based on real-time progress. Recently, the use of DRL to solve different COPs has received significant attention, as they exhibit promising outcomes in the JSSP [29]. For example, Tassel *et al*. [30] presented a DRL technique for solving the JSSP. A PPO algorithm built on an actor-critic framework was used to learn a policy network that maps a state to an action probability distribution. A related work by Hammami *et al*. [31] proposed a real-time scheduling approach based on PPO with actor-critic architecture, capable of dynamically generating schedules in response to environmental changes. These advancements underscore DRL's efficacy in tackling dynamic scheduling challenges, highlighting its relevance in the field of artificial intelligence and operations research.

Moreover, recent developments have expanded DRL applications to include the use of RNN, GNN, and transformer-based models for addressing COPs. For example, Han and Yang *et al*. [32] proposed a modified pointer network, attention mechanism, and RNN to generate scheduling solutions in real time. In another study, Fellek *et al*. [33] introduced a graph transformer combined with RL for the vehicle routing problem. This innovative approach effectively captures spatial relationships in routing scenarios, leading to improved computational efficiency and solution quality.

Chen *et al*. [14] employed a transformer-based attention mechanism coupled with a disjunctive graph embedding method for solving the JSSP. The employed attention mechanism enabled the model to learn long-range dependencies, making it suitable for solving large-scale problems. In a similar study, Gebreyesus *et al*. [34] presented an end-to-end

DRL model for solving the JSSP. The model uses an attention-based encoder of the transformer network to embed the JSSP environment represented as a disjunctive graph, and introduces a gate mechanism to enhance the representations of nodes. Recently, GNN-based approaches have also been proposed for solving COPs. These approaches leverage the structural information of the problem, and have been proven effective in solving problems with complex dependencies. Song *et al*. [35] presented a novel DRL method for flexible job shop scheduling that integrates operation selection and machine assignment, and uses a GNN-based architecture to model complex operation-machine interactions. In Park *et al*. [16], a GNN framework with RL was proposed for solving the JSSP. The framework effectively leverages the structural information present in the JSSP environment to extract rich graph representations.

Building on these foundations, our study introduces a novel approach in the form of the DRL-STGNN. While existing methods effectively address spatial relationships and dependencies within the JSSP, the lack of temporal information results in capturing incomplete graph information that fails to reflect the true dynamics and variations within scheduling environments. The DRL-STGNN is uniquely designed to capture not only the structural information through an advanced graph-based approach but also the temporal evolution of system dynamics. This dual approach ensures a comprehensive graph representation that reflects the true state of the scheduling environment. The introduction of temporal information is anticipated to enhance model performance by enabling it to capture the interdependencies between spatial and temporal features more effectively; by learning how the graph evolves over time, the model can make more informed decisions. This leads to more optimal scheduling solutions and improved overall performance. Our framework addresses both spatial and temporal dimensions; it independently learns the structural information and temporal dependencies, which are then dynamically integrated through a gate-based fusion layer. In contrast, other network structures in other domains mainly employ a simple concatenation [36] or linear projection [37] to combine the spatial and temporal embeddings. The DRL-STGNN not only enriches the representation but also enhances the overall effectiveness of the solution.

3 Problem formulation

The JSSP is defined as the problem of determining the optimal schedule of a set of n jobs on a set of m machines while adhering to a set of constraints. Each job $J_i \in \{J_1, J_2, \ldots, J_n\}$ has a given processing sequence of m_i different machines, with $m_i \leq m$. The jth operation $o_{i,j}$ of job J_i must be processed on a designated machine $M_k \in \{M_1, M_2, \ldots, M_m\}$. Each operation $o_{i,j}$ has a processing time of $p_{i,j}$. Operations must be processed without interrupting the process once it is started. The objective is to find an optimal schedule that minimizes the maximum makespan C_{max} while satisfying all constraints. The JSSP can be mathematically described as follows:

$$\min C_{max} = \max_{i,j}\{s_{i,j} + p_{i,j}\} \tag{1}$$

$$\text{s.t.} \quad s_{i,j} + p_{i,j} \leq s_{i,j+1}, \quad \forall i \in \{1, \ldots, n\}, \forall j \in \{1, \ldots, m_i - 1\} \tag{2}$$

$$\begin{aligned} &(s_{i,j} + p_{i,j} \leq s_{i',j'}) \vee (s_{i',j'} + p_{i',j'} \leq s_{i,j}), \\ &\forall i \neq i', \forall j, \forall j', \text{ if } a_{i,j,k} = a_{i',j',k} = 1 \end{aligned} \tag{3}$$

$$s_{i,j} \geq 0, \quad \forall i \in \{1, \ldots, n\}, \forall j \in \{1, \ldots, m_i\} \tag{4}$$

where $a_{i,j,k}$ is a binary variable that equals 1 if operation $o_{i,j}$ of job J_i is assigned to machine M_k, and 0 otherwise.

The objective of minimizing the makespan, as expressed in (1), is to determine the shortest possible time to complete all jobs while adhering to all operational constraints. The job constraint in (2) ensures that each operation of the same job must be processed in a specific order, dictating the sequence in which operations within a single job are executed. The machine constraint in (3) maintains that each machine can only process one operation at a time, thereby preventing the simultaneous processing of multiple operations on the same machine. Finally, the non-negativity constraint in (4) ensures that the starting time of each operation for a given job on a particular machine cannot be negative.

The scheduling problem in the JSSP is often modeled using disjunctive graphs. A disjunctive graph $G = (V, C, D)$ represents a JSSP instance, where V is the set of nodes depicting operations. C and D describe the set of conjunctive edges and disjunctive edges, respectively, referring to the different types of dependencies between operations. Two operations linked by a conjunctive edge imply that the operations belong to the same job and must be processed consecutively. In comparison, two operations connected by a disjunctive edge indicate that they need to be processed by the same machine in any order.

The process of solving the JSSP involves determining the processing sequence of the operations of the same machine. Figure 1 provides a detailed illustration of a disjunctive graph for a 3x3 JSSP instance (n x m). It sequentially depicts the stages of JSSP scheduling: (a) displays the initial state with all operations unscheduled; (b) shows an intermediate state at a specific time step t; and (c) presents the final state where every operation is scheduled, signifying the completion of the scheduling process. This

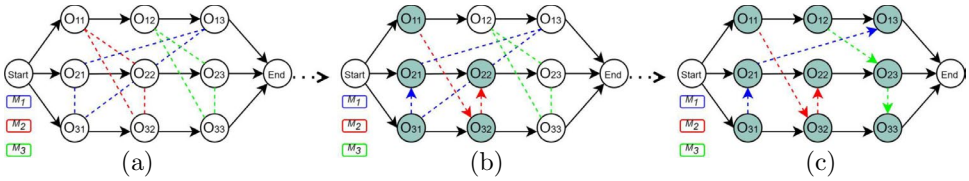


Fig. 1 Sequential stages of a JSSP instance in disjunctive graph representation. **a** Initial state with unscheduled operations; **b** Intermediate state at time step t, showing scheduled operations in light green and unscheduled in white; **c** Final state with all operations scheduled. M1, M2, and M3 denote machines linked by dashed lines of the same color for operations on the same machine

visual representation illustrates the progressive scheduling steps in the JSSP context.

The JSSP instances in different time steps are encoded as distinct state representations, which comprehensively contain the static and dynamic information of the problem at each time step. The processing time and precedence constraints of operations are static attributes of the environment, where they remain unchanged during the scheduling decision process. By contrast, the dynamic attributes, which include the operation scheduling status $R(o_{i,j}, s_t)$ and operation estimated completion time $C_E(o_{i,j}, s_t)$, change with time as the scheduling process proceeds, providing time-dependent information about the problem environment. $R(o_{i,j}, s_t)$ is a binary variable that indicates whether $o_{i,j}$ is scheduled in s_t, and $C_E(o_{i,j}, s_t)$ is given as

$$C_E(o_{i,j}, s_t) = \begin{cases} s_{i,j} + p_{i,j} & \text{if } o_{i,j} \text{ is scheduled} \\ (s_{i,j-1} + p_{i,j-1}) + p_{i,j} & \text{otherwise.} \end{cases} \quad (5)$$

We employed a reward function computed at each time step as the difference in makespans between two subsequent states, $r(s_t, a_t, s_{t+1}) = C_{max}(s_t) - C_{max}(s_{t+1})$. The cumulative sum at the end of the episode corresponds to $\sum_{t=0}^{T} r(s_t, a_t, s_{t+1}) = C_{max}(s_0) - C_{max}$, implying that maximizing rewards effectively minimizes C_{max} given the constant value of $C_{max}(s_0)$.

4 JSSP scheduling framework: DRL-STGNN

In the context of the JSSP, the actions adopted at each decision step induce the dynamics in the environment, causing changes in the topological structure of the graph and the temporal evolution of graph features. These attributes comprise spatial (structural) and temporal (time-dependent) characteristics. Although both attributes can shape the status of the job shop, existing DRL frameworks rely solely on spatial information to construct graph representations [3] [14] [35]. Owing to the presence of temporal attributes that tend to change following the actions in the environment, the entire graph becomes a dynamic JSSP graph. To account for dynamics in the temporal attributes and learn the temporal evolution of the graph features, the DRL-STGNN combines both spatial and temporal information of the JSSP environment. This allows capturing the interdependencies between the nodes and feature evolutions, enabling the extraction of expressive graph representations at each time step.

Accordingly, we designed an iterative encoder comprising three distinct blocks, namely, spatial, temporal, and fusion blocks to effectively capture the graph representations, as shown in Fig. 2. Inspired by its successful performance in solving the JSSP in [12], we built upon a GIN [38] to effectively capture the structural information of the JSSP graph in our spatial block. However, in the original implementation of the GIN, the information in the neighborhood connectivity was treated with equal weight regardless of the relative importance. Nevertheless, some of these representations are less important for the target node owing to noise, whereas others hold important information of the underlying disjunctive graph. To address this, we designed an Attentive-GIN that treats the representations in the neighborhood connectivity differently based on their importance to the target node. This allows every node of the graph to aggregate only important representations from its neighborhood connectivity as in the case of the GAT in [39]. To learn the temporal dependencies between the graph features, we constructed a temporal block comprising long short-term memory (LSTM) and feedforward network (FFN) layers, followed by a batch normalization (BN) layer.

Furthermore, to selectively integrate the spatial and temporal representations from the two blocks, we designed a novel fusion block to create a more enhanced representation of the JSSP environment. Motivated by the successful implementation of the gating mechanism for RL-based COPs in [40] [41], we introduced the gating mechanism in [42] to selectively fuse the learned representations for further processing in the decoding layer. This allows the DRL-STGNN to make more informed decisions in optimizing the scheduling of operations, resulting in improved performance of the scheduling framework.

In addition to its innovative integration of spatial and temporal features, the DRL-STGNN model is designed with computational efficiency in consideration. A key factor is its efficient node embedding process, which uses a unified

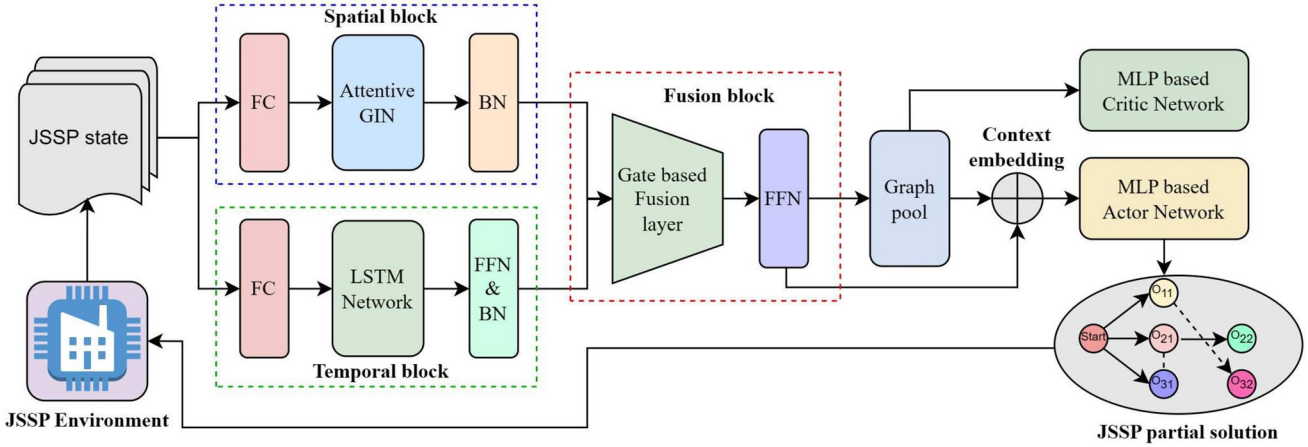


Fig. 2 End-to-end architecture of the proposed DRL-STGNN, highlighting its key components. The architecture comprises the spatial block (enclosed in a blue broken line), the temporal block (enclosed in a green broken line), and the fusion block (enclosed in a red broken line) for comprehensive scheduling

attention mechanism to integrate all relationships simultaneously, avoiding the need for multiple, relation-specific computations required in GNN implementations with distinct updater functions, such as in [16]. Unlike Transformer-based frameworks that often require extensive computational resources due to their layered complexity, DRL-STGNN uses an efficient design that minimizes computational overhead. The lightweight LSTM network for temporal information enhances model performance with minimal impact on the overall complexity. These design choices collectively contribute to the reduced computational time observed in DRL-STGNN, making it a highly effective solution for complex scheduling problems.

4.1 Spatial block

The structure of our spatial block is designed to effectively capture the structural information of the JSSP environment using a linear layer, multiple layers of Attentive-GIN, and a BN layer. The core of our spatial block, the Attentive-GIN, leverages the powerful capabilities of the GIN as proposed by [38], which is renowned for its ability to embed rich structural information of graphs. This GNN variant designed stands out for its simple structure that delivers enhanced discriminative capabilities in graph representation learning [38], proving effective in tackling scheduling challenges [12] [43].

For the implementation of the Attentive-GIN, we enhance the GIN architecture by integrating an attention mechanism, as depicted in Fig. 3. Accordingly, we first transformed the node feature vectors $X = \{x_1, x_2, \ldots, x_n\}$ from the JSSP environment using a linear layer to obtain initial node representations $P^{(0)} = \{p_1^{(0)}, p_2^{(0)}, \ldots, p_n^{(0)}\}$, where $P^{(0)}$ indicates the initial representation of nodes, with superscript (0) denoting the layer index starting from the input and n representing the total number of nodes.

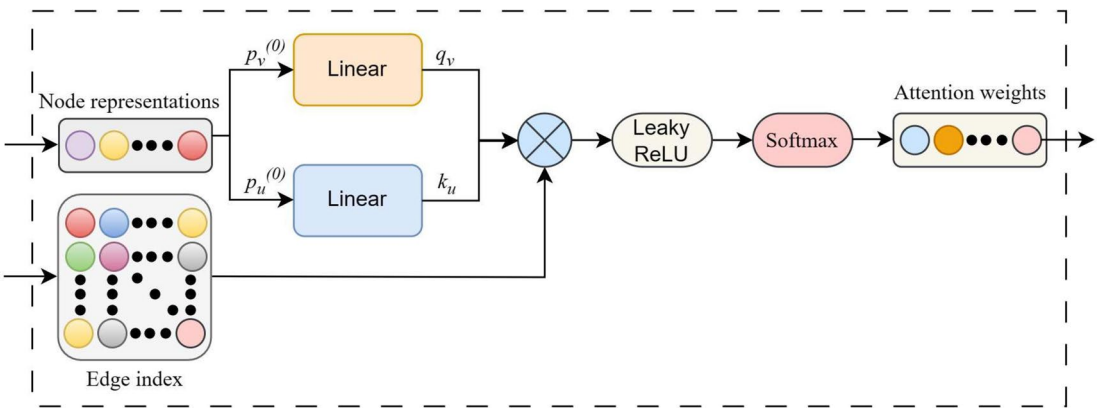


Fig. 3 Attention mechanism for aggregating neighborhood information within the Attentive-GIN-based spatial block

Next, for each node $p_v^{(0)}$ in the input, we constructed a query vector q_v and key vector k_v.

$$q_v = p_v^{(0)} W^Q \in \mathbb{R}^{n \times d_q}$$
$$k_v = p_v^{(0)} W^K \in \mathbb{R}^{n \times d_k} \tag{6}$$

where W^Q and W^K are learnable matrices, d_q and d_k represent the dimensions of the query and key vectors respectively.

To obtain the similarity e_{vu} between a given node v and each node u in its neighborhood connectivity $N_{(v)}$, we performed a dot product multiplication $(\cdot)$ between the query vector q_v and each key vector k_u transposed (k_u^T). The adjacency matrix was introduced into the computation to incorporate the graph structure, ensuring that node v only aggregated the features of nodes within $N_{(v)}$. Then, a LeakyReLU activation was applied to the resulting values.

$$e_{vu} = \text{LeakyReLU}(q_v \cdot {k_u}^T). \tag{7}$$

We then passed the obtained similarity e_{vu} to the Softmax layer to obtain a normalized attention coefficient α_{vu}.

$$\alpha_{vu} = \frac{exp(e_{vu})}{\sum_{w \in N_{(v)}} exp(e_{vw})} \quad \forall\, w \in N_{(v)}. \tag{8}$$

The computed attention weight implies the degree of importance of each node to all other nodes in the neighborhood connections. We used this attention to update the node representations by aggregating the context information encoded in $N_{(v)}$ based on their importance in a similar fashion to that in the popular attention mechanisms in graph networks [39]. The aggregation process is performed recursively for K layers, enabling the node representation to leverage the structural information contained in the connectivity of graph nodes within K hops. For each layer $k \in K$, the aggregated representations are then fed to the MLP layer to produce the kth node representation update p_v^k.

$$p_v^{(k)} = MLP_{\theta_k}^{(k)}\left(\alpha_{vv} * p_v^{(k-1)} + \sum_{u \in N_{(v)}} \alpha_{vu} * p_u^{(k-1)}\right). \tag{9}$$

This process sequentially refines the node features through the network layers, with the resultant features further normalized by a BN layer before proceeding to the subsequent processing stages in the fusion block.

4.2 Temporal block

Unlike most DRL-based models to solve graph-structured COPs that make an episodic encoding of graph attributes, our encoder encodes the node attributes at each decision time step t to capture the changing attributes. The change in attributes after every decision renders a dynamic disjunctive graph. To adapt to this change, the DRL-STGNN integrates the evolving information into the graph's overall structure as additional information of the graph. This enables the DRL-STGNN to fully capture the dynamics induced in the environment following the actions at each decision time step, allowing it to generate a complete graph representation.

To effectively encode these dynamics, the temporal block incorporates an LSTM network specifically designed to capture the temporal dependencies that arise as graph attributes change over time. This capability is essential for adapting to the dynamic nature of the graph where node attributes vary significantly with each decision step. The LSTM layer is complemented by FFN, which introduces non-linearity to uncover complex patterns and relationships in the data. Subsequently, BN layer is applied to stabilize the training process by normalizing the inputs to each layer, thereby enhancing model performance and expediting convergence. This composition within the temporal block is meticulously designed to not only preserve the integrity of temporal data but also to refine the learning and decision-making processes in dynamic environments.

To this end, the newly obtained node features $X(t) = \{x_1(t), x_2(t), \ldots, x_n(t)\}$ for $t \in \{1, 2, \ldots, T\}$ undergo a linear transformation and are encoded into the LSTM layer to capture their temporal dependency at each decision time step t. This process is repeated until all operations are scheduled, allowing the temporal evolution of the time-varying node features of the JSSP environment to be fully captured over the scheduling time horizon T. Finally, the learned representations are passed to the FFN layer followed by the BN layer, allowing improved and stable training of the deep networks.

$$P_\tau(t) = \text{BN}(\text{FFN}(\text{LSTM}(W \cdot X(t) + b))) \tag{10}$$

where W and b are learnable weight matrices.

4.3 Fusion block

To obtain an expressive representation, we designed a novel fusion block comprising a gating mechanism and FFN layer to capture the complex relationship between the representations obtained from the previous two blocks. In effect, we employed a gate-based layer to selectively combine the learned spatial representations $P_s(t) = \{p_1^{(K)}(t), p_2^{(K)}(t), \ldots, p_n^{(K)}(t)\}$ and temporal representations $P_\tau(t) = \{p_1(t), p_2(t), \ldots, p_n(t)\}$. This layer allows the model to dynamically learn to combine the representations by assigning weights based on their importance. We used the gating mechanism in [42] to leverage its increased performance in capturing a long-term dependency. With both the spatial and temporal aspects of the JSSP

selectively integrated, the resulting spatio-temporal representation $P_{s\tau}^{(t)}$ is more representative of the dynamics in the environment.

In the context of a JSSP with time-varying features, the gate units in the gating mechanism adapt to the changing environment as the scheduling process progresses. Additionally, the gating mechanism prevents the problem of vanishing gradients by suppressing noise features. This enables the model to identify and focus on important features, producing more expressive representations. Then, $P_{s\tau}^{(t)}$ is fed to the FFN layer to enhance further and refine the representations and yield an improved feature extraction. The FFN layer comprises one hidden layer and a ReLU activation function.

$$P_{s\tau}^{(t)}(P_\tau(t), P_s(t)) = \mathrm{FFN}((1 - z_t) \odot P_s(t) + z_t \odot h_t) \tag{11}$$

where the hidden state h_t, reset gate r_t, and update gate z_t within the gating mechanism are given as follows:

$$\begin{aligned} h_t &= \tanh(W_g P_\tau(t) + U_g(r_t \odot P_s(t))) \\ r_t &= \sigma(W_r P_\tau(t) + U_r P_s(t)) \\ z_t &= \sigma(W_z P_\tau(t) + U_z P_s(t) - b_g) \end{aligned} \tag{12}$$

where W_g, U_g, W_r, U_r, W_z, and U_z are weight matrices of linear projections, $\sigma(\cdot)$ is the sigmoid function, $\odot$ represents element-wise multiplication, and $b_g = 0.1$.

4.4 Pointer-based decoding layer

This layer sequentially maps the learned representations from the encoder layer into an optimal action at every time step t. As in existing works [12] [34], we employed a stack of MLP layers in the decoding layer. We used a Softmax-based pointer layer to select an operation from a set of candidate jobs at every time step. To this end, we first employed a graph pooling mechanism and obtained a global representation of the graph $p_{graph}^{(t)}$ from the updated node representations $P_{s\tau}^{(t)}$. Our graph pooling mechanism effectively aggregates the overall graph information by averaging the node representations using (13). The resulting graph representation provides a comprehensive representation of the entire graph as it effectively reflects the information contained within the graph.

$$p_{graph}^{(t)} = \frac{1}{n} \sum_{i \in n} P_{s\tau}^{(t)}. \tag{13}$$

Next, as shown in (14), a context embedding vector $C_{s\tau}^{(t)}$ was constructed from the outputs of the encoder layer by concatenating the updated node representations $P_{s\tau}^{(t)}$ and global graph representation $p_{graph}^{(t)}$ from (11) and (13). This process generates a comprehensive context embedding that captures both individual node features and the overall graph information. We then passed the obtained context embedding $C_{s\tau}^{(t)}$ into a stack of MLP layers, followed by a Softmax layer, to obtain the probability distribution $P(a_t)$ of candidate operations. During training, we used the sampling algorithm to select the next operation to be processed based on $P(a_t)$.

$$C_{s\tau}^{(t)} = (P_{s\tau}^{(t)} \| p_{graph}^{(t)}). \tag{14}$$

$$P(a_t) = Softmax(MLP_{\theta_\pi}(C_{s\tau}^{(t)})). \tag{15}$$

4.5 DRL training strategy

Our DRL model is trained using the PPO algorithm to leverage its sample efficiency and stability during training [16]. Our PPO follows an actor-critic framework where the actor is the policy network π_θ that learns to select actions based on the current state of the environment, and the critic network v_ϕ estimates the values of selected actions. The algorithm iteratively updates the θ and ϕ parameters of the actor and critic networks, respectively, using a policy gradient approach.

In the PPO algorithm, the actor network's training hinges on a clipped surrogate objective and entropy-based regularization. The clipped surrogate objective function, $L^{\mathrm{CLIP}}(\theta)$, combines the advantage function, $\hat{A}_t$, given in (17), with the probability ratio, $r_t(\theta)$, given in (18). A clipping function, defined by the clipping parameter ϵ, constrains the range of $r_t(\theta)$ to ensure stability.

$$L^{\mathrm{CLIP}}(\theta) = \sum_t \min\left(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t\right). \tag{16}$$

The advantage function $\hat{A}_t$ is computed as the difference between the discounted sum of rewards and the baseline estimate at the state. This calculation of the generalized advantage estimate is less computationally demanding than methods requiring Q-function estimates, as it avoids evaluating all possible future actions and states, focusing instead on direct reward outcomes.

$$\hat{A}_t = \sum_t \gamma^t r_t - v_\phi(s_t). \tag{17}$$

The probability ratio $r_t(\theta)$ compares the likelihood of selecting an action under the current policy against the old policy:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}. \tag{18}$$

The entropy regularization, $L^E(\theta)$, as shown in (19), reinforces exploration to maintain a diverse range of actions, avoiding premature convergence to sub-optimal deterministic strategies.

$$L^{E}(\theta) = \sum_{t} \text{Entropy}\left(\pi_{\theta}(a_t|s_t)\right). \tag{19}$$

The critic network employs a mean squared error loss for value function estimation:

$$L^{\text{VF}}(\phi) = \sum_{t} \left(v_{\phi}(s_t) - \left(\sum_{t} \gamma^{t} r_t \right) \right)^{2}. \tag{20}$$

Finally, the combined objective function of PPO integrates the clipped surrogate objective, value function loss, and entropy regularization in a weighted manner. It is formulated as:

$$L(\theta, \phi) = c_p L^{\text{CLIP}}(\theta) - c_v L^{\text{VF}}(\phi) + c_e L^{E}(\theta) \tag{21}$$

where c_p, c_v, and c_e are the respective coefficients assigned to the clipped surrogate objective, value function loss, and entropy regularization. The training procedure is comprehensively described in Algorithm 1.

5 Experiments and results

This section discusses the experimental settings used to train and test the DRL-STGNN and the results obtained by implementing it on different JSSP instances. The settings are similar for all JSSP instances solved by the DRL-STGNN. The performance of the DRL-STGNN was compared with that of heuristic dispatching rules and learning-based solutions. Specifically, we selected four established heuristics: shortest processing time (SPT), most work remaining (MWKR), most operations remaining (MOPNR), and minimum ratio of flow due date to most work remaining (FDD/MWKR) [44] [12].

Additionally, we conducted comparative experiments between the proposed DRL-STGNN and existing DRL-based solutions such as GIN [12], gated-attention model (GAM) [34], GNN-L2S [16], and disjunctive graph embedded recurrent decoding transformer (DGERD) [14].

Algorithm 1 Training procedure with PPO

Require: DRL-STGNN, number of epochs for update E, PPO steps I, number of independent actors for update N, trainable parameters of actor-network θ, behavioral actor θ_{old}, and critic network $\varnothing$.

Initialization: initialize parameter sets of π_{θ}, $\pi_{\theta_{\text{old}}}$, and $v_{\varnothing}$.
for iter $= 1$ to I **do**
  Pick N independent JSSP instances from distribution D.
  for $n = 1$ to N **do**
    for $t = 0, 1, 2, \ldots$ **do**
      while $s_{n,t}$ is not terminal **do**
        Sample $a_{n,t}$ based on $\pi_{\theta_{\text{old}}}$.
        Obtain reward $r_{n,t}$ and next state $s_{n,t+1}$.
        Compute the advantage function $\hat{A}_{n,t}$ and probability ratio $r_{n,t}(\theta)$.
      end while
    end for
  end for
  Calculate the policy loss $L_n^{\text{CLIP}}(\theta)$, the value function loss $L_n^{\text{VF}}(\phi)$, and the entropy loss $L_n^{E}(\theta)$.
  Compute the PPO loss $L_n(\theta, \varnothing)$.
  for $e = 1, 2, \ldots, E$ **do**
    Update network parameters $\theta, \varnothing$.
    $\theta, \varnothing \leftarrow \arg\max \left(\sum_{n=1}^{N} L_n(\theta, \varnothing) \right)$.
  end for
  Update θ_{old}.
end for
Output: Trained parameter set of θ.

A brief overview of the baselines used for comparison is given below.

- SPT: prioritizes a job with the shortest processing time.
- MWKR: prioritizes a job with most work remaining.
- MOPNR: prioritizes a job with most operations remaining.
- FDD/MWKR: prioritizes a job with the minimum ratio of flow due date to most work remaining.
- GIN: GNN-based scheduling approach integrated with DRL.
- GAM: DRL framework that incorporates a transformer attention model enhanced with a gate mechanism.
- GNN-L2S: A JSSP scheduling framework leveraging GNN and DRL.
- DGERD: DRL framework utilizing the transformer attention mechanism and disjunctive graph embedding.
- Google OR-Tools: an exact solver based on constraint programming.

5.1 Experimental settings and hyperparameters

In this study, we leveraged generated instances from the JSSP for both training and testing, following the methodology detailed in [45] and [12]. We investigated seven distinct problem sizes, as presented in Table 2, with the operation processing times $p_{i,j}$ uniformly distributed within $U[1, 99]$. We trained our models on small to medium instance sizes: 6x6, 10x10, 15x15, and 20x20, with subsequent testing on their respective instance sizes. The training times for these instance sizes were 1.8, 3.6, 8.2, and 26.3 h, respectively. In all the experiments, the DRL-STGNN was trained once for 10,000 iterations with 100 validation instances, followed by performance evaluation on 100 randomly selected instances. To assess the generalization capabilities of the DRL-STGNN, training data from the 20x20 instance size were utilized to test against the larger instances (30x20, 50x20, and 100x20).

In addition to the generated instances, we also assessed the performance using public JSSP benchmarks, including the Taillard's [46] and DMU benchmarks [47], to determine DRL-STGNN's generalizability and suitability for real-world production scenarios. During testing, we used a greedy algorithm to select unscheduled operations from the candidate operations.

Table 1 Hyperparameters

Parameters	Value
Node embedding dimension d_h	128
Number of FFN units N_{FFN}	512
Number of GIN convolution layers L_{GIN}	3
Recursive aggregation layers K	2
Number of LSTM layers L_{LSTM}	4
Policy loss coefficient c_p	2
Value loss coefficient c_v	1
Entropy loss coefficient c_e	0.01
Discount factor γ	1.0
Learning rate decay α_{decay}	0.9
Clipping parameter ϵ	0.2
PPO learning rate η_{PPO}	1×10^{-5}
PPO epochs E	1
Optimizer	Adam

To establish a performance benchmark for our generated test instances, we employed Google OR-Tools,[1] a widely recognized exact solver that derives precise solutions using constraint programming. Additionally, in line with [12], we adopted the best-known solutions from the literature[2] as reference benchmarks for the Taillard's and DMU instances.

To compare the performance of the DRL-STGNN to that of the baselines, we used the average makespan C_{max}, computation time, and optimality gap as performance metrics. The optimality gap was calculated with (22), using the best solution C^*_{max} for each instance as a comparison benchmark. The average makespan C_{max} and computation times (in seconds) from 10 trials are summarized in Tables 2, 3, and 4 for the generated, Taillard's, and DMU datasets, respectively. The corresponding optimality gaps are illustrated in Figs. 4, 5 and 6. As the DGERD code is not available online, our assessment of the DGERD was based on the reported results for Taillard's benchmark. Our code was based on PyTorch 1.7 and Python 3.8, and all the experiments were conducted on a single RTX GPU. The hyperparameters given in Table 1 were used for all the experiments.

$$Opt.\ Gap = \left(\frac{C_{\max} - C^*_{\max}}{C^*_{\max}} \right) \times 100\% \tag{22}$$

[1] [Online]. Available: https://developers.google.com/optimization.

[2] [Online]. Best solutions are available at: http://optimizizer.com/TA.php for Taillard's instances and http://optimizizer.com/DMU.php for DMU instances.

Table 2 Comparison of the DRL-STGNN and the baselines in terms of makespan and computation time on randomly generated datasets. Bold values represent the minimum C_{max} and Time for each instance size

		Heuristic dispatching rules				Learning-based solutions			
Size		SPT	MWKR	MOPNR	FDD/ MWKR	GIN	GAM	GNN-L2S	DRL-STGNN
6x6	C_{max}	691.9	656.9	604.6	630.2	574.1	569.3	567.1	**552.5**
	Time	**0.01**	**0.01**	**0.01**	**0.01**	0.06	0.2	0.22	0.1
10x10	C_{max}	1210.9	1151.4	1102.9	1101.1	988.6	985.7	**955.1**	977.9
	Time	**0.03**	0.04	0.04	0.04	0.18	0.54	0.67	0.27
15x15	C_{max}	1890.9	1812.1	1722.7	1693.3	1504.8	1506.7	1497.4	**1490.9**
	Time	**0.11**	0.12	**0.11**	0.12	0.44	1.57	3.49	0.61
20x20	C_{max}	2519.8	2469.2	2328.1	2263.7	2007.8	2026.1	1992.3	**1990.2**
	Time	**0.3**	0.31	0.31	0.31	0.93	2.85	13.24	1.13
30x20	C_{max}	3208.7	3080.1	2883.9	2809.6	2508.3	2507.2	2485.7	**2476.2**
	Time	**0.72**	0.73	0.73	0.74	1.8	3.9	57.3	2.11
50x20	C_{max}	4469.8	4273.1	3993.4	3859.1	3522.5	3494.8	**3411.8**	3471.6
	Time	**2.5**	2.52	**2.5**	2.52	4.87	9.98	301.41	4.98
100x20	C_{max}	7516.1	7069.7	6658.2	6385.3	6088.7	5988.8	**5948.3**	5981.7
	Time	16.66	16.69	**16.63**	16.72	28.62	34.94	3018.32	29.2

5.2 Performance on generated instances

In this section, we compare the performance of the DRL-STGNN against the selected baselines, which include heuristic dispatching rules and learning-based solutions. Table 2 presents the performance comparison in terms of C_{max} and the computational efficiency. The results show that the DRL-STGNN successfully outperformed both the heuristic dispatching rules and learning-based solutions with good solution quality and reasonable computational time. While the heuristic dispatching rules consistently achieved faster run times for all instance sizes, their lack of guarantee for optimal results limits their effectiveness.

DRL-STGNN, in contrast, outperformed most DRL baselines in terms of computational efficiency, with GIN being the sole exception where a slight increase in run time was observed. This slight increase in run time can be attributed to our use of the attention mechanism for neighborhood information aggregation in the DRL-STGNN, as opposed to the GIN, which employs a simpler summation-based approach. However, the enhanced performance of the DRL-STGNN compensates for the slight time discrepancy observed with the GIN.

In comparison to the best rule from the above heuristics, performance improvements of 9.4%, 12.6%, 13.6%, and 13.7% in C_{max} are achieved for 6x6, 10x10, 15x15, and 20x20 instance sizes, respectively. For the larger instances of 30x20, 50x20, and 100x20, similar performance improvements of 13.5%, 11.2%, and 6.7%, respectively, are observed. It is worth noting that although the DRL-STGNN is trained for smaller-sized instances, it is capable of processing larger-sized instances while maintaining performance on C_{max} and computation time. This performance scalability is evident in the results presented in Table 2 and optimality gap depicted in Fig. 4. Additionally, the DRL-STGNN maintained this performance on the learning-based solutions by achieving performance gains ranging from 0.9% to 3.9% and 0.1% to 3.1% over the competitor models, GIN and GAM, respectively, across all the instance sizes.

When compared to GNN-L2S, DRL-STGNN demonstrated a performance improvement of 0.1% to 3% across 6x6, 15x15, 20x20, and 30x20 instance sizes, despite GNN-L2S having significantly longer computation times. However, GNN-L2S demonstrated better makespan performance by 0.6% to 2.8% on the 10x10, 50x20, and 100x20 instances despite longer computation times, whereas DRL-STGNN achieved competitive results with a simpler design and significantly reduced computation times. The primary advantage of the DRL-STGNN lies in its ability to capture and integrate both spatial and temporal features of the JSSP.

This integration generally enhances the performance of the model by providing a more comprehensive representation of the scheduling environment. However, the complexity of effectively learning and leveraging these features can vary depending on the specific characteristics of the dataset, as different instances exhibit distinct structural and dynamic properties with varying importance of spatial and temporal attributes. Although DRL-STGNN consistently demonstrated better performance compared to the other baselines on all instances, GNN-L2S showed better optimality performance on certain instances. We believe that not consistently outperforming GNN-L2S is due to the inherent characteristics of the instances which make the temporal information of some instances less significant.

As can be observed in the optimality gap graph, the gap initially starts small and increases up to the 30x20 instance size, but subsequently decreases for the larger instances. This pattern was also observed in [12] and [45], suggesting

Table 3 Performance Comparison of the DRL-STGNN against the baselines on Taillard's benchmark

	Heuristic dispatching rules						Learning-based solutions									
	SPT		MWKR		MOPNR		FDD/MWKR		DGERD		GIN		GAM		GNN-L2S		DRL-STGNN	
Instance	C_{max}	Time	C_{max}	Time	C_{max}	Time	C_{max}	Time	C_{max}	Time	C_{max}	Time	C_{max}	Time	C_{max}	Time	C_{max}	Time
Tai (15x15)	1902.6	**0.11**	1927.5	0.12	1782.3	**0.11**	1808.6	0.12	1675.2	9.8	1547.4	0.4	1530.1	1.26	**1499.1**	4.15	1525.3	1.13
Tai (20x15)	2253.6	**0.17**	2190.7	0.18	2015.8	**0.17**	2054	0.18	1799	12.1	1774.7	0.6	1797.9	1.92	**1724.1**	10.86	1763.3	1.87
Tai (20x20)	2655.8	0.31	2518.6	0.31	2309.9	**0.3**	2387.2	0.31	2080.8	16.4	2128.1	0.9	2086.3	2.78	2088.6	16.76	**2064.3**	2.42
Tai (30x15)	2888.4	**0.38**	2728	0.39	2601.3	**0.38**	2590.8	0.39	2420.6	20.4	2378.8	1.1	2342.3	3.12	2341.3	42.35	**2328.6**	3.06
Tai (30x20)	3234.7	**0.72**	3193.3	0.73	2888.1	**0.72**	3045	0.73	2651.7	27.5	2603.9	1.8	2603.5	3.83	2592	67.65	**2586**	4.31
Tai (50x15)	4194.7	**1.21**	3907.8	1.22	3608	**1.21**	3736.3	1.23	3636.9	50	3393.8	2.7	3343.8	5.37	**3280.1**	241.38	3328.7	5.01
Tai (50x20)	4532.2	**2.5**	4375.1	2.53	3920	2.51	4022.1	2.52	3672.9	64.3	3593.9	4.9	3534	9.9	3531.7	389.5	**3525.6**	7.87
Tai (100x20)	7564.6	16.65	7128.8	16.69	6452.3	**16.64**	6620.7	16.75	6276.7	273.1	6097.6	28.3	6027.1	39.12	**5951.1**	3980.1	6016.5	37.21

Bold values represent the minimum C_{max} and time for each instance size

that JSSPs with larger job-to-machine ratios (|*J*|/|*M*|) are easier to solve, although larger instances remain hard for exact solvers owing to the inherent NP-hardness of the JSSP. In this regard, the relation-specific node embedding structure of the GNN-L2S, coupled with the fact that it is trained with sampling instance sizes, performed better on certain instances, especially those with a smaller average optimality gap. The DRL-STGNN, trained on smaller fixed-sized instances, focuses on both the spatial and temporal information which generally aids in capturing richer representations, but can also result in slightly less optimal performance on certain instances.

In terms of computation time, the DRL-STGNN achieved significantly reduced time compared to the GNN-L2S while maintaining competitive optimality. This is mainly attributed to the simpler design of the DRL-STGNN implementation. The GNN-L2S handles node embeddings and message passing using distinct updater functions for different types of relationships (precedent, succedent, disjunctive), leading to a more granular and relation-specific update process. This requires separate computations for each relationship type, followed by concatenation of the results, making the process more computationally intensive. Additionally, GNN-L2S employs more iterations (higher *K*) for node embedding, allowing for higher-order interactions but increasing computational demands. In contrast, DRL-STGNN employs a unified attention mechanism that handles all relationships simultaneously, dynamically weighting the importance of neighboring nodes through attention coefficients. While incorporating a temporal block, the DRL-STGNN utilizes a lightweight LSTM-based network, which adds minimal complexity to the overall design. This integrated approach allows DRL-STGNN to update node embeddings in a single, more streamlined pass, reducing the need for multiple, relation-specific computations and thus offering greater computational efficiency.

Overall, DRL-STGNN demonstrated competitive solution quality with reduced computation time, underscoring its superior efficiency and overall solution quality in handling the complex dependencies and dynamic nature of the JSSP. The observed performance enhancement underscores the effectiveness of considering the temporal aspect of the JSSP in constructing a rich graph representation. By effectively constructing an iterative encoder that embeds temporal and spatial representations at each time step, the model learns to capture the dynamics emerging in the JSSP following the decision at each time step. Additionally, the gating mechanism employed to fuse the learned representation plays a crucial role by selectively fusing the spatial representations from the spatial block and the temporal attributes captured by the temporal block, as discussed in Sect. 4.

Table 4 Performance Comparison of the DRL-STGNN against the baselines on DMU benchmark

	Heuristic dispatching rules								Learning-based solutions					
	SPT		MWKR		MOPNR		FDD/MWKR		GIN		GAM		DRL-STGNN	
Instance	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time	$C_{\max}$	Time
DMU (20x15)	4951.5	0.33	4909.9	0.33	4513.2	**0.31**	4666.3	0.34	4215.3	0.85	4209.5	2.19	**4051.3**	1.58
DMU (20x20)	5690.5	**0.46**	5489.0	0.47	5052.3	0.47	5298.2	0.48	4804.5	1.21	4782.7	3.24	**4687.4**	2.25
DMU (30x15)	6306.2	**0.52**	6252.9	0.54	5742.8	**0.52**	6016.5	0.55	5557.9	1.48	5533.8	3.85	**5415.4**	2.74
DMU (30x20)	7036.0	**0.71**	6925.0	0.72	6491.9	0.72	6827.3	0.72	5967.4	1.81	5983.6	5.46	**5859.3**	3.59
DMU (40x15)	7601.2	**0.74**	7484.2	0.75	7105.5	**0.74**	7420.0	0.79	6636.5	1.97	6646.8	5.51	**6615.6**	3.66
DMU (40x20)	8538.1	**1.31**	8460.9	1.33	7870.7	**1.31**	8210.9	1.33	7354.5	2.89	7372.7	7.89	**7340.7**	5.38
DMU (50x15)	8975.4	**1.20**	8906.0	1.21	8436.5	1.22	9150.2	1.22	8179.4	2.69	8182.1	7.33	**8117.4**	5.03
DMU (50x20)	10132.8	**2.50**	9807.0	2.52	9408.0	2.52	9899.6	2.52	8725.9	4.91	8809.3	9.88	**8717.4**	7.30

Bold values represent the minimum C_{max} and time for each instance size

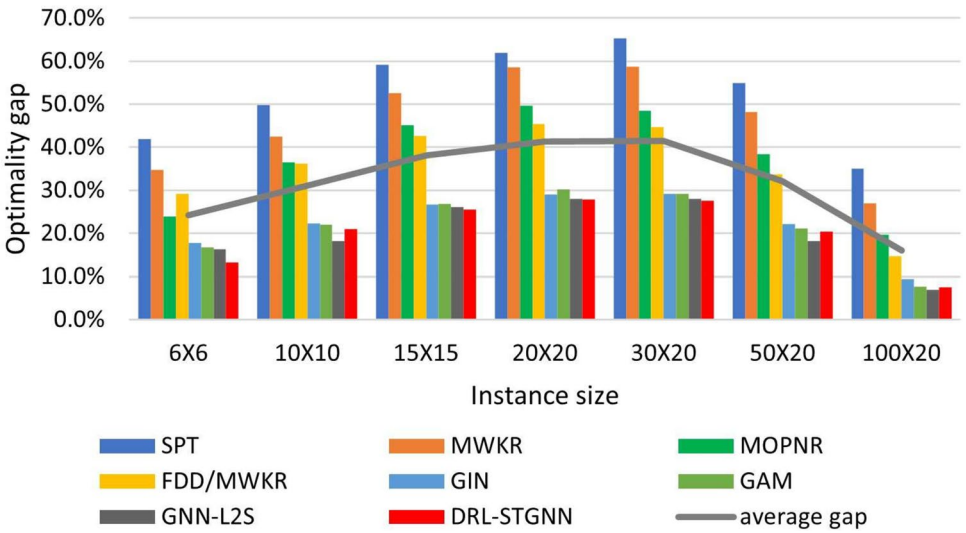


Fig. 4 Comparison of the DRL-STGNN with the heuristic dispatching rules and learning-based solutions based on optimality gap for generated instances. See Sect. 5.2 for the explanation on the trend

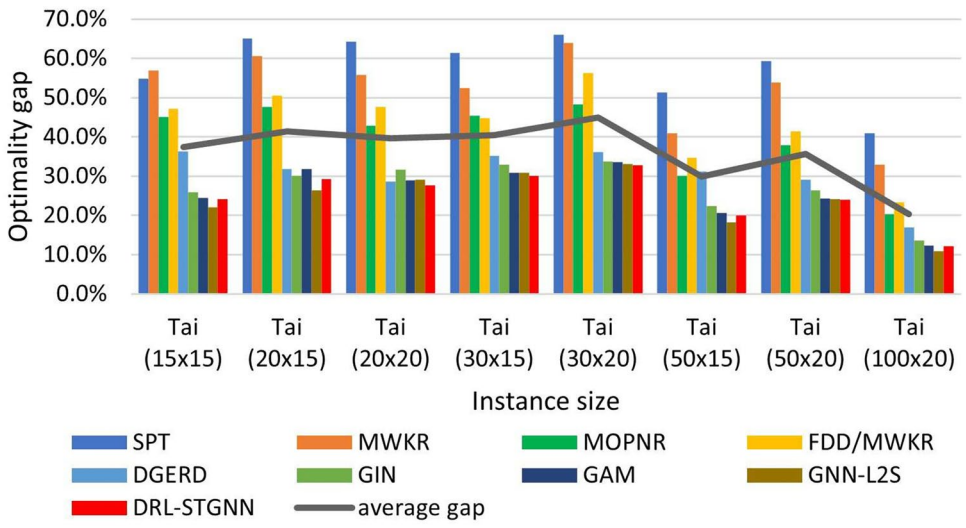


Fig. 5 Comparison of the DRL-STGNN with the heuristic dispatching rules and learning-based models based on optimality gap for Taillard's instances

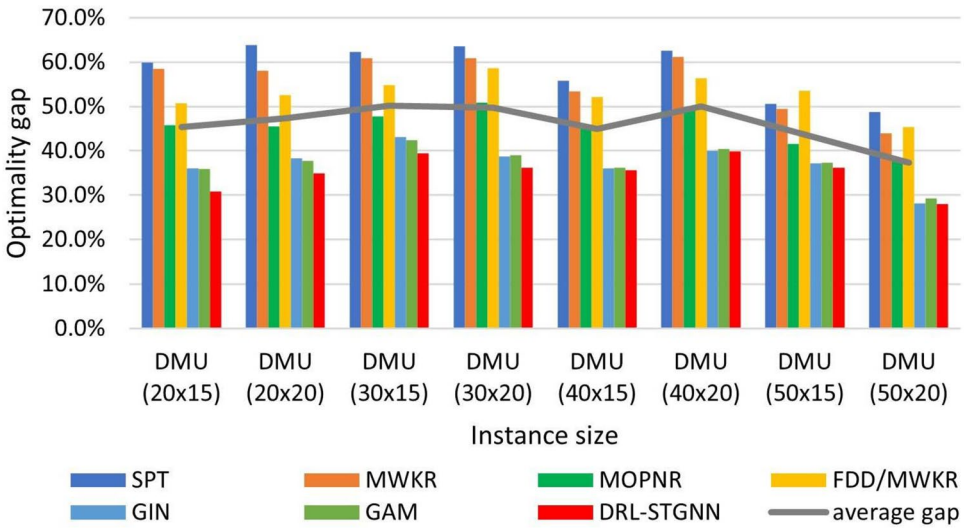


Fig. 6 Comparison of the DRL-STGNN with the heuristic dispatching rules and learning-based models based on optimality gap for DMU instances

5.3 Performance on public test benchmarks

In addition to the randomly generated test instances, we conducted comparative experiments on standard JSSP benchmarks, including the Taillard's and DMU datasets. For the Taillard's benchmark, we conducted experiments on all eight groups of instances covering from Tai (15x15) to Tai (100x20), each with ten instances. We employed models trained on 15x15, 20x15, 20x20, and 30x15 problem sizes for testing, each on their respective sizes. Furthermore, we utilized the 30x15 size model to evaluate larger instances ranging from Tai (30x20) to Tai (100x20). Similarly, for the DMU benchmark, we tested models trained on 20x15, 20x20, and 30x15 problem sizes on their corresponding instances. We also used the 30x15 model for larger cases, ranging from DMU (30x20) to DMU (50x20). Table 3 presents a comparison of the results of the DRL-STGNN on Taillard's benchmarks to those of the learning-based and heuristic solutions.

Similarly to the generated test instances, for Taillard's test instances, the DRL-STGNN demonstrated significant improvement compared to heuristic solutions and most learning-based baselines in terms of C_{max} across all problem instances while maintaining reasonable computation times. Furthermore, Fig. 5 highlights DRL-STGNN's reduced optimality gap compared to heuristic solutions and most learning-based solutions, demonstrating its competitive performance. Notably, the optimality gap was more noticeably reduced for problem sizes larger than Tai (30x20), a trend that is consistent with that in the results for generated data instances. This performance advantage of the DRL-STGNN is also demonstrated on DMU benchmark instances. Table 4 and Fig. 6 present the performance improvements achieved by the DRL-STGNN over the baseline models across the DMU test benchmarks, highlighting its competitive performance. It is noteworthy that the DRL-STGNN shows efficient scheduling times on public benchmarks, outperforming most learning-based solutions, as can be observed in Tables 3 and 4.

To further enrich the comprehensiveness of our performance evaluation, we incorporated two additional metaheuristic algorithms, the genetic algorithm (GA) [48] and migrating birds optimization (MBO) algorithm [49], into our assessment of the DRL-STGNN on public benchmarks. The GA, a well-known optimization technique inspired by principles of natural evolution, alongside MBO, a novel metaheuristic that mimics the V-formation flight patterns of migrating birds, were both selected owing to their extensive application and proven success in addressing complex

Table 5 Computation time (in seconds) comparison

Instance size	DRL-STGNN	GA	MBO
Tai (15x15)	1.13	131.77	1260.63
Tai (20x15)	1.87	177.45	1717.67
Tai (20x20)	2.42	241.86	2280.54
Tai (30x15)	3.06	272.76	2559.90
Tai (30x20)	4.31	391.58	3567.96
Tai (50x15)	5.01	536.29	4734.92
Tai (50x20)	7.87	739.69	7015.72
Tai (100x20)	37.21	2154.95	20439.47
DMU (20x15)	1.58	179.78	1755.43
DMU (20x20)	2.25	239.11	2234.11
DMU (30x15)	2.74	275.19	2514.34
DMU (30x20)	3.59	383.29	3403.75
DMU (40x15)	3.66	400.03	3648.86
DMU (40x20)	5.38	568.17	4964.51
DMU (50x15)	5.03	505.85	4792.15
DMU (50x20)	7.30	734.49	7028.22

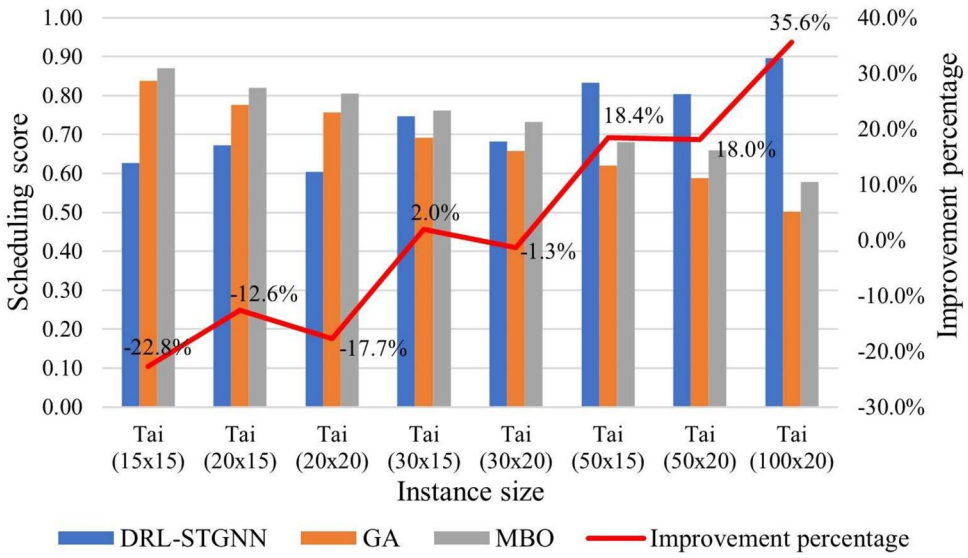


Fig. 7 Comparative experimental results of the DRL-STGNN and other well-known algorithms on 8 groups of Taillard's dataset instances with different sizes

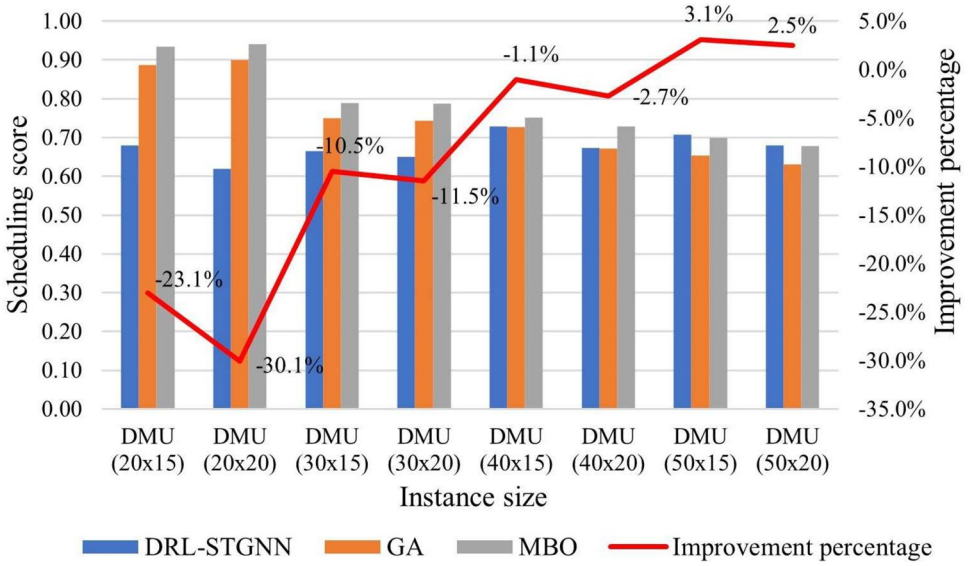


Fig. 8 Comparative experimental results of the DRL-STGNN and other well-known algorithms on 8 groups of DMU dataset instances with different sizes

scheduling problems [23] [50] [51] [52]. Based on [53] and [14], we express the results using the scheduling score: scheduling score $= C_{LB}/C_{max}$, where C_{LB} is the instance's lower bound and C_{max} is the algorithm's minimal makespan.

As depicted in Fig. 7, using Taillard's benchmark, the GA and MBO outperformed the DRL-STGNN for the Tai (15x15), Tai (20x15), and Tai (20x20) instances. However, for larger instances from Tai (30x15) to Tai (100x20), the DRL-STGNN demonstrated superior performance, as indicated by the red broken performance improvement line. Notably, whereas GA's performance began to decline after Tai (20x20), MBO maintained its superiority up to Tai (30x20) before exhibiting a steady drop. In Fig. 8, a similar trend is observed: both the GA and MBO perform well up to DMU 30x20, after which the DRL-STGNN begins to outperform the other algorithms. This analysis reveals that, compared to the metaheuristic algorithms GA and MBO, DRL-STGNN performs well on larger instances, while GA and MBO demonstrate better performance on smaller problem settings. The improved performance of GA and MBO on smaller scales may be due to their simpler and more adaptable frameworks, which are better suited to these dimensions compared to the complexities in the architecture or parameter settings of DRL-STGNN.

The enhanced performance of DRL-STGNN on larger instances, compared to the conventional metaheuristics, can be attributed to its distinct architectural components. First, the foundation of DRL allows for dynamic adaptability in vast search domains, proving invaluable for intricate JSSPs [54]. Second, the integration of an attention mechanism

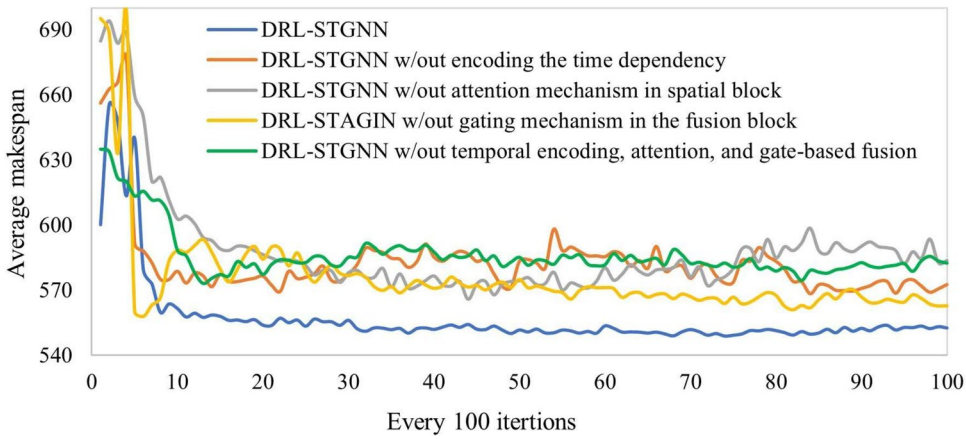


Fig. 9 Ablation study on the contributions of proposed modifications in the DRL-STGNN

ensures that critical long-term dependencies are effectively captured, thus optimizing the model's decision-making precision [55]. Third, the encoder structure within the DRL-STGNN systematically embeds the temporal evolution of node features, ensuring a complete and comprehensive graph representation that accurately captures the complexities of the JSSP environment. Collectively, these aspects underscore the proficiency of the DRL-STGNN over conventional metaheuristics in handling larger scheduling instances.

Additionally, the data in Table 5 demonstrate that the search-based methods, GA and MBO, require significantly higher computation times compared to that of the DRL-STGNN, despite its high-quality solutions. This underscores the effectiveness of the DRL-STGNN in generating solutions with reduced computational demand.

Generally, the results consistently show that DRL-STGNN delivers superior performance across a wide range of JSSP problems, with reduced computation time, from randomly generated instances to established public benchmarks. This highlights its effectiveness in tackling diverse JSSP challenges without retraining, making it a robust solution for various problem sets. The fact that the DRL-STGNN captures both spatial and temporal attributes of the environment at every time step produces more expressive graph representations, which in turn give rise to good generalization to different types of JSSP instances unseen during training and outperform the baselines with better solution quality. It is also worth noting that in addition to maintaining the stability of training, the PPO implemented in this study and the quality of the PPO hyperparameters selected for training were also conducive to the superior performance of the DRL-STGNN.

5.4 Ablation study

To show the contribution of each component of the DRL-STGNN, we performed an ablation study for JSSP 6x6. We used the same experimental setup and hyperparameters as those described in Sect. 5.1 for both training and testing. In the ablation study, we evaluated the performance of the DRL-STGNN with and without the proposed modifications. Accordingly, we demonstrated the result in Fig. 9.

The ablation study demonstrates that the DRL-STGNN, incorporating the key modifications, achieves performance improvements, fast convergence, and maintains stability, as shown in the figure. Without encoding the time dependency of the JSSP, the performance of the model significantly dropped, highlighting the missing necessary information to build the graph representation. The removal of the attention mechanism from the Attentive-GIN caused a similar performance drop. Further, the implementation of the DRL-STGNN without a gating mechanism in the fusion block tended to make the training unstable in addition to the performance drift. Therefore, the gating mechanism used to selectively integrate the representations from the spatial and temporal blocks enabled the model to use only important representations in learning the scheduling of operations in the policy network. Moreover, removing the temporal encoding, attention mechanism for spatial attribute extraction, and gate-based fusion from the DRL-STGNN resulted in decreased sample efficiency and reduced performance. Overall, the ablation study verified that the performance gain was due to the key components in the DRL-STGNN.

6 Conclusion

In this study, a DRL framework called DRL-STGNN is proposed to solve the JSSP. Unlike existing learning-based frameworks that disregard the time-dependent information of the JSSP over the decision-making steps, the DRL-STGNN effectively captures both the spatial and temporal attributes of the JSSP, ensuring the expression ability of the entire disjunctive graph. The encoder of the DRL-STGNN mainly comprises spatial, temporal, and fusion blocks. The

Attentive-GIN in the spatial block attentively aggregates neighborhood information to capture the structural dependency between nodes. Similarly, the temporal block iteratively embeds the temporal evolution of the node features over the scheduling decision steps to learn the time-dependent attributes of the JSSP. Subsequently, we applied a gate mechanism-based fusion block to effectively combine the learned representations from the two blocks based on their importance at each time step. Then, we employed a simple stack of MLP layers to sequentially decode optimal schedules. We trained the DRL-STGNN using an actor-critic-based PPO algorithm, and conducted extensive comparative experiments to evaluate the performance. The results show that the DRL-STGNN outperforms heuristics and learning-based solutions on both random instances and public JSSP benchmarks.

In a future study, we propose extending the implementation of the DRL-STGNN for solving other COPs by adapting the model to handle problem-specific inputs, constraints, and objectives. This extension would involve exploring alternative neural network architectures and fusion mechanisms, as well as incorporating domain-specific knowledge and features into the learning process.

Acknowledgements This research is supported by the Kaizen Doctoral Program, under the Japan International Cooperation Agency (JICA).

Author contributions **Goytom Gebreyesus:** Conceptualization, Methodology, Software, Data curation, Formal analysis, Writing—original draft, Writing—review & editing; **Getu Fellek:** Conceptualization, Data curation, Writing—original draft, Writing—review & editing; **Ahmed Farid:** Conceptualization, Data curation, Writing—original draft, Writing—review & editing; **Sicheng Hou:** Conceptualization, Data curation, Visualization, Writing - original draft, Writing—review & editing; **Shigeru Fujimura:** Conceptualization, Methodology, Data curation, Investigation, Writing—original draft, Supervision, Project administration; **Osamu Yoshie:** Conceptualization, Methodology, Data curation, Writing—original draft, Supervision, Project administration.

Data availability The data that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Mezgebe TT, Bril El Haouzi H, Demesure G, Pannequin R, Thomas A (2020) Multi-agent systems negotiation to deal with dynamic scheduling in disturbed industrial context. J Intell Manuf 31:1367–1382
2. Sime DM, Wang G, Zeng Z, Wang W, Peng B (2022) Semi-supervised defect segmentation with pairwise similarity map consistency and ensemble-based cross-pseudo labels. IEEE Trans Ind Inform 19(9):9535–9545
3. Qiao D, Duan L, Li H, Xiao Y (2023) Optimization of job shop scheduling problem based on deep reinforcement learning. Evolut Intell 17(1):371–383
4. Zhang Z, Zheng L, Weng MX (2007) Dynamic parallel machine scheduling with mean weighted tardiness objective by q-learning. Int J Adv Manuf Technol 34:968–980
5. Shahrabi J, Adibi MA, Mahootchi M (2017) A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. Comput Ind Eng 110:75–82
6. Kardos C, Laflamme C, Gallina V, Sihn W (2021) Dynamic scheduling in a job-shop production system with reinforcement learning. Procedia CIRP 97:104–109
7. Zhao Y, Wang Y, Tan Y, Zhang J, Yu H (2021) Dynamic jobshop scheduling algorithm based on deep q network. IEEE Access 9:122995–123011
8. Zhang M, Lu Y, Hu Y, Amaitik N, Xu Y (2022) Dynamic scheduling method for job-shop manufacturing systems by deep reinforcement learning with proximal policy optimization. Sustainability 14(9):5177
9. Zhao L, Shen W, Zhang C, Peng K (2022) An end-to-end deep reinforcement learning approach for job shop scheduling. In: 2022 25th international conference on computer supported cooperative work in design (CSCWD), pp. 841-846. IEEE. https://doi.org/10.1109/CSCWD54268.2022.9776116
10. Bello I, Pham H, Le QV, Norouzi M, Bengio S (2016) Neural combinatorial optimization with reinforcement learning. arXiv preprint arXiv:1611.09940
11. Nazari M, Oroojlooy A, Snyder L, Takác M (2018) Reinforcement learning for solving the vehicle routing problem. Advances in neural information processing systems **31**
12. Zhang C, Song W, Cao Z, Zhang J, Tan PS, Chi X (2020) Learning to dispatch for job shop scheduling via deep reinforcement learning. Adv Neural Inform Process Syst 33:1621–1632
13. Yang S (2022) Using attention mechanism to solve job shop scheduling problem. In: 2022 2nd international conference on consumer electronics and computer engineering (ICCECE), pp. 59–62. IEEE
14. Chen R, Li W, Yang H (2022) A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job-shop scheduling problem. IEEE Trans Ind Inform 19(2):1322–1331
15. Lv M, Hong Z, Chen L, Chen T, Zhu T, Ji S (2020) Temporal multi-graph convolutional network for traffic flow prediction. IEEE Trans Intell Transp Syst 22(6):3337–3348
16. Park J, Chun J, Kim SH, Kim Y, Park J (2021) Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. Int J Prod Res 59(11):3360–3377
17. Wagner HM (1959) An integer linear-programming model for machine scheduling. Naval Res Logist Q 6(2):131–140
18. Pinson E (1995) The job shop scheduling problem: A concise survey and some recent developments. Sched Theory Appl 93:277–294
19. Kurniawan B, Song W, Weng W, Fujimura S (2021) Distributed-elite local search based on a genetic algorithm for bi-objective

job-shop scheduling under time-of-use tariffs. Evolut Intell 14:1581–1595
20. Mohan J, Lanka K, Rao AN (2019) A review of dynamic job shop scheduling techniques. Procedia Manuf 30:34–39
21. Hou S, Gebreyesus GD, Fujimura S (2024) Day-ahead multimodal demand side management in microgrid via two-stage improved ring-topology particle swarm optimization. Expert Syst Appl 238:122135
22. Gao H, Kwong S, Fan B, Wang R (2014) A hybrid particle-swarm tabu search algorithm for solving job shop scheduling problems. IEEE Trans Ind Inform 10(4):2044–2054
23. Yamada T, Nakano R (1997) Genetic algorithms for job-shop scheduling problems. Proceedings of modern heuristic for decision support **6781**
24. Cunha B, Madureira A, Fonseca B, Matos J (2021) Intelligent scheduling with reinforcement learning. Appl Sci 11(8):3710
25. Sur C (2019) Ucrlf: unified constrained reinforcement learning framework for phase-aware architectures for autonomous vehicle signaling and trajectory optimization. Evolut Intell 12(4):689–712
26. Ding S, Du W, Zhao X, Wang L, Jia W (2019) A new asynchronous reinforcement learning algorithm based on improved parallel pso. Appl Intell 49:4211–4222
27. Zhang L, Zhang Y, Liu S, Chen L, Liang X, Cheng G, Liu Z (2024) Orad: a new framework of offline reinforcement learning with q-value regularization. Evolut Intell 17(1):339–347
28. Zhao X, Ding S, An Y, Jia W (2019) Applications of asynchronous deep reinforcement learning based on dynamic updating weights. Appl Intell 49:581–591
29. Kayhan B, Yildiz G (2021) Reinforcement learning applications to machine scheduling problems: a comprehensive literature review. J Intell Manuf. https://doi.org/10.1007/s10845-021-01847-3
30. Tassel P, Gebser M, Schekotihin K (2021) A reinforcement learning environment for job-shop scheduling
31. Hammami NEH, Lardeux B, Hadj-Alouane A, Jridi M (2022) Job shop scheduling: A novel drl approach for continuous schedule-generation facing real-time job arrivals. IFAC-PapersOnLine 55, 2493–2498 https://doi.org/10.1016/j.ifacol.2022.10.083
32. Han B, Yang J (2021) A deep reinforcement learning based solution for flexible job shop scheduling problem. Int J Simul Modell 20(2):375–386
33. Fellek G, Farid A, Gebreyesus G, Fujimura S, Yoshie O (2023) Graph transformer with reinforcement learning for vehicle routing problem. IEEJ Trans Electr Electron Eng 18(5):701–713
34. Gebreyesus G, Fellek G, Farid A, Fujimura S, Yoshie O (2023) Gated-attention model with reinforcement learning for solving dynamic job shop scheduling problem. IEEJ Trans Electr Electron Eng 18(6):932–944
35. Song W, Chen X, Li Q, Cao Z (2022) Flexible job-shop scheduling via graph neural network and deep reinforcement learning. IEEE Trans Ind Inform 19(2):1600–1610
36. Gunarathna U, Borovica-Gajic R, Karunasekara S, Tanin E (2022) Solving dynamic graph problems with multi-attention deep reinforcement learning. arXiv preprint arXiv:2201.04895
37. Wu G, Zhang Z, Liu H, Wang J (2021) Solving time-dependent traveling salesman problem with time windows with deep reinforcement learning. In: 2021 IEEE international conference on systems, man, and cybernetics (SMC), pp 558–563. IEEE
38. Xu K, Hu W, Leskovec J, Jegelka S (2018) How powerful are graph neural networks? arXiv preprint arXiv:1810.00826
39. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2017) Graph attention networks. arXiv preprint arXiv:1710.10903
40. Xu Y, Fang M, Chen L, Xu G, Du Y, Zhang C (2021) Reinforcement learning with multiple relational attention for solving vehicle routing problems. IEEE Trans Cybern. https://doi.org/10.1109/TCYB.2021.3089179
41. Guo F, Wei Q, Wang M, Guo Z, Wallace SW (2023) Deep attention models with dimension-reduction and gate mechanisms for solving practical time-dependent vehicle routing problems. Transp Res Part E: Logist Transp Rev 173:103095
42. Parisotto E, Song F, Rae J, Pascanu R, Gulcehre C, Jayakumar S, Jaderberg M, Kaufman RL, Clark A, Noury S, et al (2020) Stabilizing transformers for reinforcement learning. In: International conference on machine learning, pp. 7487–7498. PMLR
43. Elsayed EK, Elsayed AK, Eldahshan KA (2022) Deep reinforcement learning-based job shop scheduling of smart manufacturing. CMC-Comput Mater Contin 73(3):5103–5120
44. Sels V, Gheysen N, Vanhoucke M (2012) A comparison of priority rules for the job shop scheduling problem under different flow time-and tardiness-related objective functions. Int J Prod Res 50(15):4255–4270
45. Taillard E (1993) Benchmarks for basic scheduling problems. Eur J Op Res 64(2):278–285
46. Jain AS, Meeran S (1999) Deterministic job-shop scheduling: Past, present and future. Eur J Op Res 113(2):390–434
47. Demirkol E, Mehta S, Uzsoy R (1998) Benchmarks for shop scheduling problems. Eur J Op Res 109(1):137–141
48. Holland JH (1992) Adaptation in Natural and Artificial Systems: an Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. MIT press, Cambridge, MA
49. Duman E, Uysal M, Alkaya AF (2012) Migrating birds optimization: a new metaheuristic approach and its performance on quadratic assignment problem. Inform Sci 217:65–77
50. Bhatt N, Chauhan NR (2015) Genetic algorithm applications on job shop scheduling problem: A review. In: 2015 International conference on soft computing techniques and implementations (ICSCTI), pp 7–14. IEEE
51. Meng T, Pan Q-K, Li J-Q, Sang H-Y (2018) An improved migrating birds optimization for an integrated lot-streaming flow shop scheduling problem. Sw Evolut Comput 38:64–78
52. Li H, Zhu H, Jiang T (2020) Modified migrating birds optimization for energy-aware flexible job shop scheduling problem. Algorithms 13(2):44
53. Han B-A, Yang J-J (2020) Research on adaptive job shop scheduling problems based on dueling double dqn. IEEE Access 8:186474–186495. https://doi.org/10.1109/ACCESS.2020.3029868
54. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. Nature 518(7540):529–533
55. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł (2017) Polosukhin, I.: Attention is all you need. Advances in neural information processing systems **30**

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.